



Slovenská technická univerzita v Bratislave  
Fakulta informatiky a informačných technológií

# Protokol k zadaniu

DBS — Databázové technológie

Zadanie 3: Hudobná streamovacia služba

**Študent:** Dominik Jurkas, Filip Hromada  
**Študijná skupina:** Streda 12:00, Ing. A. Tomčala  
**Akademický rok:** 2025/2026  
**Dátum:** 3. mája 2026

# Obsah

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Zmeny oproti zadaniu 1</b>                                           | <b>2</b>  |
| 1.1      | Odstránenie tabuľky Library . . . . .                                   | 2         |
| 1.2      | Doplnené CHECK obmedzenia . . . . .                                     | 2         |
| 1.3      | Doplnené UNIQUE obmedzenia . . . . .                                    | 3         |
| 1.4      | Pridané DEFAULT hodnoty . . . . .                                       | 4         |
| 1.5      | Stratégie ON UPDATE a ON DELETE . . . . .                               | 4         |
| 1.6      | Drobné technické úpravy . . . . .                                       | 4         |
| 1.7      | Biznis pravidlá nepokryté na úrovni DDL . . . . .                       | 4         |
| <b>2</b> | <b>Fyzický model – implementácia</b>                                    | <b>5</b>  |
| 2.1      | Zoznam tabuliek . . . . .                                               | 5         |
| 2.2      | Zaujímavé integritné obmedzenia . . . . .                               | 5         |
| 2.3      | Zvolené ON DELETE stratégie . . . . .                                   | 6         |
| <b>3</b> | <b>Seed dáta</b>                                                        | <b>7</b>  |
| 3.1      | Nástroj a postup generovania . . . . .                                  | 7         |
| 3.2      | Počty záznamov v jednotlivých tabuľkách . . . . .                       | 7         |
| 3.3      | Zvolená distribúcia . . . . .                                           | 7         |
| 3.4      | Zabezpečenie konzistencie . . . . .                                     | 8         |
| <b>4</b> | <b>Proces 1 – Správa queue</b>                                          | <b>9</b>  |
| 4.1      | Popis procesu a väzba na Scenár 4 zo Zadania 1 . . . . .                | 9         |
| 4.2      | Validácie zabezpečené databázou . . . . .                               | 9         |
| 4.3      | SQL implementácia . . . . .                                             | 9         |
| 4.4      | Ukážka spustenia nad seed dátami . . . . .                              | 10        |
| 4.5      | Indexy podporujúce Proces 1 . . . . .                                   | 12        |
| <b>5</b> | <b>Proces 2 – Rebríček popularity skladieb a artistov</b>               | <b>12</b> |
| 5.1      | Popis procesu . . . . .                                                 | 12        |
| 5.2      | Použité SQL techniky . . . . .                                          | 12        |
| 5.3      | SQL implementácia . . . . .                                             | 13        |
| 5.4      | Ukážka spustenia nad seed dátami . . . . .                              | 14        |
| 5.5      | Zdôvodnenie korektnosti . . . . .                                       | 15        |
| 5.6      | Indexy podporujúce Proces 2 . . . . .                                   | 15        |
| <b>6</b> | <b>Výkonnosť a indexy</b>                                               | <b>15</b> |
| 6.1      | Zoznam vytvorených indexov . . . . .                                    | 15        |
| 6.2      | Zdôvodnenie indexov . . . . .                                           | 16        |
| 6.3      | EXPLAIN ANALYZE --pred a po pridaní idx_playlist_song_song_id . . . . . | 17        |
| 6.4      | Pomalé dopyty aj s indexmi . . . . .                                    | 18        |
| <b>7</b> | <b>Zhodnotenie – reflexia pracovného procesu</b>                        | <b>18</b> |
| 7.1      | Čo nás prekvapilo . . . . .                                             | 18        |
| 7.2      | Chyba v pôvodnom modeli . . . . .                                       | 19        |
| 7.3      | Najťažšie biznis pravidlo na vyjadrenie v SQL . . . . .                 | 19        |
| 7.4      | Použitie AI nástrojov . . . . .                                         | 19        |
| 7.5      | Pracovný proces . . . . .                                               | 20        |

# 1 Zmeny oproti zadaniu 1

Počas implementácie fyzického modelu sme vykonali viacero úprav voči pôvodnému návrhu zo Zadania 1. Tieto zmeny boli motivované predovšetkým spätnou väzbou cvičiaceho, požiadavkami fyzického modelu (premietnutie biznis pravidiel priamo do databázy) a poznatkami získanými počas implementácie procesov.

## 1.1 Odstránenie tabuľky Library

Najvýznamnejšou štrukturálnou zmenou bolo odstránenie tabuľky **Library**. V pôvodnom návrhu reprezentovala knižnicu používateľa, do ktorej patrili jeho playlisty. Vzťah **User** - **Library** bol typu **1:1**, pričom samotná knižnica neniesla žiadny hlbší význam okrem odkazu na používateľa, Bola skôr ako príprava pre budúcnosť kedy sme chceli pridať podcasty do našej platformy.

**Dopad na ostatné tabuľky:**

- Tabuľka **Playlist** pôvodne odkazovala na **Library** cez stĺpec `library_id`. Tento bol nahradený stĺpcom `user_id` odkazujúcim priamo na **User**.
- Vzťah **User** - **Playlist** je teraz priamo **1:N**, čo zjednodušuje SQL dopyty (úspora jedného JOINu pri každom prístupe k playlistom používateľa).
- Obmedzenie unikátnosti názvu playlistu v rámci knižnice bolo presunuté na **UNIQUE** (`user_id`, `name`) v tabuľke **Playlist**, čím je zachovaná pôvodná biznis logika.

## 1.2 Doplnené CHECK obmedzenia

V Zadaní 1 boli mnohé biznis pravidlá popísané iba slovne („názov nemôže byť prázdny“). Vo fyzickej implementácii sme ich premietli do explicitných **CHECK** obmedzení, aby ich vynucovala databáza, nie aplikácia.

### Neprázdne textové polia

Pre stĺpce `Role.name`, `User.name`, `Device.name`, `Advertisement.name`, `Advertisement.content`, `Song.name`, `Queue.name` a `Playlist.name` sme pridali obmedzenie:

```
1 CHECK (LENGTH(TRIM(name)) > 0)
```

Samotné **NOT NULL** totiž zabráni iba hodnote **NULL**, ale prázdny reťazec `''` alebo reťazec zložený z medzier by prešiel. Tieto by inak spôsobili nezmyselné záznamy (napr. „playlist bez mena“).

### Validácia formátu emailu

```
1 CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
```

V pôvodnom návrhu bol `email` **UNIQUE**, čo síce zabezpečí jedinečnosť, ale negarantuje validný formát. Bez tohto obmedzenia by databáza prijala napríklad reťazec `asdf` ako platný email.

### Trvanie skladby

```
1 CHECK (duration > 0 AND duration <= 7200)
```

Pôvodný návrh definoval iba `duration INT NOT NULL`. Skladba s nulovým, záporným alebo extrémne dlhým trvaním nemá zmysel. Hornú hranicu 7200 sekúnd (2 hodiny) sme zvolili ako rozumný limit pre hudobnú skladbu.

## Časová pečiatka zobrazenia reklamy

```
1 CHECK (shown_at <= CURRENT_TIMESTAMP + INTERVAL '1_minute')
```

Záznam o zobrazení reklamy nemôže vzniknúť v budúcnosti. Tolerancia jednej minúty pokrýva drobné odchýlky systémových hodín pri distribuovaných systémoch.

## Kladné pozície vo fronte a playliste

```
1 CHECK (queue_position > 0)
2 CHECK (position > 0)
```

Pozícia s hodnotou 0 alebo zápornou je nezmyselná. Pôvodný návrh túto kontrolu neobsahoval.

## Minimálna dĺžka hesla

```
1 CHECK (LENGTH(password) >= 8)
```

Hoci je v stĺpci `password` typu `VARCHAR(255)` typicky uložený hash, tento check chráni pred chybou v aplikácii, ktorá by uložila prázdny alebo extrémne krátky reťazec.

## 1.3 Doplnené UNIQUE obmedzenia

### Prevenca duplicít v queue a playlistoch

```
1 UNIQUE (queue_id, song_id)
2 UNIQUE (playlist_id, song_id)
```

Zadanie 1 v Scenári 4 explicitne uvádzalo: „Ak skladba už v queue existuje a systém nepovoľuje duplicity, operácia sa nevykoná.” a v sekcii Konkrétne obmedzenia: „V jednom playliste sa rovnaká skladba nemôže nachádzať viackrát.”

V pôvodnom návrhu boli tieto pravidlá iba odporúčané. Vo fyzickom modeli ich vynucujeme priamo cez `UNIQUE` obmedzenia, takže duplicita zlyhá už na úrovni databázy.

### Unikátne pozície

```
1 UNIQUE (queue_id, queue_position)
2 UNIQUE (playlist_id, position)
```

Dve rôzne skladby nemôžu vo fronte ani playliste sedieť na rovnakej pozícii. Bez tohto obmedzenia by sa mohli vytvárať nedefinované usporiadané kolekcie. Toto obmedzenie zároveň ovplyvnilo implementáciu **Procesu 1** (správa queue), kde sme museli použiť 4-krokový algoritmus pre zmenu poradia (viď kapitola Proces 1).

### Unikátne názvy playlistov v rámci používateľa

```
1 UNIQUE (user_id, name)
```

Po odstránení tabuľky `Library` muselo byť toto pravidlo presunuté priamo na `Playlist`. Zadanie 1 v Scenári 1 spomínalo: „Systém skontroluje, či v danej knižnici už neexistuje playlist s rovnakým názvom.”

| Stĺpec                            | DEFAULT           | Dôvod                                     |
|-----------------------------------|-------------------|-------------------------------------------|
| Device.is_active                  | TRUE              | Nové zariadenie je aktívne                |
| Advertisement.is_active           | TRUE              | Nová reklama je aktívna                   |
| Advertisement_Impression.shown_at | CURRENT_TIMESTAMP | Aktuálny čas pri INSERTe                  |
| Song.is_approved                  | FALSE             | Implementuje biznis pravidlo zo Scenáru 2 |

Tabuľka 1: Pridané DEFAULT hodnoty

## 1.4 Pridané DEFAULT hodnoty

V niektorých prípadoch sme doplnili rozumné východiskové hodnoty:

Najdôležitejší je posledný riadok: `Song.is_approved` DEFAULT FALSE priamo implementuje pravidlo zo Zadania 1 (Scenár 2): „Po nahratí skladby je skladba uložená ako neschválená a čaká na kontrolu administrátora.”

## 1.5 Stratégie ON UPDATE a ON DELETE

V Zadaní 1 boli stratégie pre ON DELETE popísané. Vo fyzickej implementácii sme ich rozšírili aj o ON UPDATE CASCADE, ktoré v pôvodnom návrhu neboli explicitne uvedené.

**Zachované zásady zo Zadania 1:**

- Role → User: ON DELETE RESTRICT – nedá sa zmazať rola, ktorú niekto má
- Song.author\_id → User: ON DELETE RESTRICT – chráni autorské väzby
- Všetky ostatné FK: ON DELETE CASCADE

**Pridané ON UPDATE CASCADE** pre všetky cudzie kľúče. V PostgreSQL sa hodnoty SERIAL typicky nemenia, ale toto je lacná poisťka pre prípad migračných skriptov, ktoré by hodnoty primárnych kľúčov menili.

## 1.6 Drobné technické úpravy

**Úvodzovky okolo User**

V PostgreSQL je `user` rezervované kľúčové slovo. Pri použití bez úvodzoviek by `CREATE TABLE User` skončilo syntaktickou chybou. Pre tabuľku používateľov sme preto použili identifikátor v dvojitých úvodzovkách `User`, čo zachováva zamýšľaný názov entity.

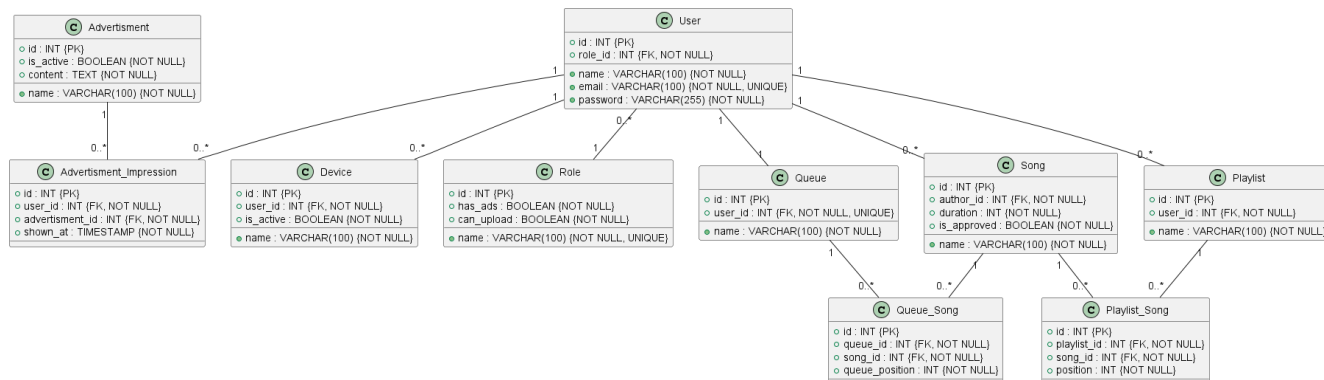
## 1.7 Biznis pravidlá nepokryté na úrovni DDL

Niektoré biznis pravidlá zo Zadania 1 **nie je možné v čistom DDL** vynútiť bez triggerov, ktoré Zadanie 3 zakazuje. Tieto pravidlá zostávajú na strane procesov v `queries.sql` alebo aplikačnej logiky:

- **Iba ARTIST môže byť autor skladby** – vyžadovalo by trigger alebo CHECK so subquery, ktoré PostgreSQL nepodporuje. Validuje sa v seed skripte (kde sa explicitne vyberajú iba ARTIST používatelia ako autori) a v procesoch nahrávania skladby.
- **Skladba pridávaná do playlistu/queue musí byť is\_approved = TRUE** – rovnaký dôvod. Validuje sa v Procese 1 podmienkou `WHERE s.is_approved = TRUE`.
- **Reklamu vidí iba používateľ s rolou has\_ads = TRUE** – validuje sa pri INSERT do `Advertisement_Impression`.

## 2 Fyzický model – implementácia

V tejto kapitole popisujeme implementáciu fyzického modelu v PostgreSQL. Schéma obsahuje 10 tabuliek (vrátane väzobných), ktoré reprezentujú všetky entity hudobnej streamovacej služby zo Zadania 1.



Obr. 1: UML diagram

### 2.1 Zoznam tabuliek

| Tabuľka                  | Popis                                                                     |
|--------------------------|---------------------------------------------------------------------------|
| Role                     | Typy používateľov (USER, PREMIUM_USER, ARTIST, ADMIN) s ich oprávneniami. |
| User                     | Všetci používatelia systému (bežní, premium, artisti, admini).            |
| Device                   | Zariadenia, na ktorých sú používatelia prihlásení.                        |
| Advertisement            | Globálne reklamy v systéme.                                               |
| Advertisement_Impression | Záznamy o zobrazeniach reklám konkrétnym používateľom.                    |
| Song                     | Hudobné skladby nahrávané používateľmi s rolou ARTIST.                    |
| Queue                    | Fronta prehrávania (1 na používateľa).                                    |
| Queue_Song               | Väzobná tabuľka medzi Queue a Song (M:N) s pozíciou.                      |
| Playlist                 | Playlisty používateľov.                                                   |
| Playlist_Song            | Väzobná tabuľka medzi Playlist a Song (M:N) s pozíciou.                   |

Tabuľka 2: Zoznam tabuliek vo fyzickom modeli

### 2.2 Zaujímavé integritné obmedzenia

V tejto sekcii zhrňame netriviálne obmedzenia, ktoré chránia kľúčové biznis pravidlá zo Zadania 1.

#### CHECK obmedzenia

- CHECK (email ~\* '...email regex...') – garantuje že hodnota emailu má platný formát. Bez tohto obmedzenia by UNIQUE constraint prijal aj nezmyselné reťazce.

- CHECK (duration > 0 AND duration <= 7200) – skladba musí mať trvanie v rozsahu 1 sekunda až 2 hodiny. Chráni pred záznamami s nulovým, záporným alebo extrémne dlhým trvaním.
- CHECK (shown\_at <= CURRENT\_TIMESTAMP + INTERVAL '1 minute') – záznam o zobrazení reklamy nemôže vzniknúť v budúcnosti.
- CHECK (LENGTH(TRIM(name)) > 0) – aplikované na všetky textové názvy. Zabraňuje prázdny alebo medzerovým reťazcom, ktoré by NOT NULL sám neodfiltroval.
- CHECK (queue\_position > 0) a CHECK (position > 0) – pozícia v queue/playliste nemôže byť 0 ani záporná.

#### UNIQUE obmedzenia

- UNIQUE (queue\_id, song\_id) a UNIQUE (playlist\_id, song\_id) – zabraňuje duplicitnému výskytu skladby v rámci jednej queue/playlistu. Toto pravidlo zo Zadania 1 („V jednej playliste sa rovnaká skladba nemôže nachádzať viackrát”) je týmto vynútené priamo databázou.
- UNIQUE (queue\_id, queue\_position) a UNIQUE (playlist\_id, position) – v rámci jednej queue/playlistu nemôžu dve skladby zdieľať rovnakú pozíciu. Toto obmedzenie zásadne ovplyvnilo implementáciu Procesu 1 (správa queue), kde sme museli použiť 4-krokový algoritmus pre zmenu poradia.
- UNIQUE (user\_id, name) v tabuľke Playlist – používateľ nemôže mať dva playlisty s rovnakým názvom.
- UNIQUE (user\_id) v tabuľke Queue – realizuje vzťah 1:1 medzi používateľom a jeho frontou.

#### Cudzie kľúče (FK)

Všetky cudzie kľúče sú deklarované s explicitnou stratégiou ON DELETE a ON UPDATE CASCADE. Neexistuje žiadne NULL-able FK, čo zaručuje že každý záznam vždy referencuje existujúcu nadradenú entitu.

### 2.3 Zvolené ON DELETE stratégie

| Vzťah                         | Stratégia | Zdôvodnenie                                                                    |
|-------------------------------|-----------|--------------------------------------------------------------------------------|
| User → Role                   | RESTRICT  | Nesmie sa zmazať rola, ktorú niekto má. Najprv sa musia presunúť používatelia. |
| Song → User (autor)           | RESTRICT  | Chráni autorské väzby. Bez autora skladba stráca legitimitu.                   |
| Device → User                 | CASCADE   | Bez používateľa nemajú zariadenia zmysel.                                      |
| Queue → User                  | CASCADE   | Queue je súčasťou identity používateľa.                                        |
| Playlist → User               | CASCADE   | Playlisty patria používateľovi.                                                |
| Advertisement_Impression → *  | CASCADE   | História patrí svojim entitám.                                                 |
| Queue_Song, Playlist_Song → * | CASCADE   | Väzobné záznamy nemajú zmysel bez svojich strán.                               |

Tabuľka 3: Stratégie ON DELETE

Kľúčové rozhodnutie je **RESTRICT** pre vzťah **Role** → **User**. Pri **CASCADE** by zmazanie role napríklad **USER** viedlo k zmazaniu všetkých bežných používateľov, čo je nebezpečné. **RESTRICT** vynúti, aby sa najprv vyriešili závislosti.

## 3 Seed dáta

### 3.1 Nástroj a postup generovania

Na generovanie testovacích dát sme použili **Python skript** (`generate_seed.py`) využívajúci knižnicu **Faker** pre realistické mená a textové dáta. Skript generuje SQL súbor (`seed.sql`) s **INSERT** príkazmi.

Hlavné technické rozhodnutia:

- **Fixný seed** (`random.seed(42)`) – zaručuje reprodukovateľnosť výstupu. Každé spustenie skriptu vyprodukuje identické dáta.
- **TRUNCATE na začiatku** – skript je idempotentný, môže sa spúšťať opakovane.
- **Batch INSERTy po 1000 riadkoch** – PostgreSQL spracuje veľké INSERTy efektívnejšie ako tisícky jednoriadkových.
- **Explicitné ID-čka** v INSERToch + následné `SELECT setval(...)` – zabezpečuje, že sequence sa nastaví na správnu hodnotu pre budúce inserty cez aplikáciu.
- **Vážená náhodnosť** (`random.choices` s váhami) – nepoužili sme uniformnú distribúciu, ale power-law segmenty pre realistický biznisový charakter dát.

### 3.2 Počty záznamov v jednotlivých tabuľkách

| Tabuľka                  | Počet záznamov |
|--------------------------|----------------|
| Role                     | 4              |
| User                     | 2 000          |
| Device                   | 3 276          |
| Advertisement            | 50             |
| Advertisement_Impression | 8 000          |
| Song                     | 3 000          |
| Queue                    | 2 000          |
| Queue_Song               | 9 215          |
| Playlist                 | 2 616          |
| Playlist_Song            | 23 915         |
| <b>SPOLU</b>             | <b>54 076</b>  |

Tabuľka 4: Počty seedovaných záznamov

Celkový objem 54 076 záznamov **päťnásobne prevyšuje** minimálnu požiadavku 10 000.

### 3.3 Zvolená distribúcia

Distribúciu sme zvolili podľa **power-law modelu**, ktorý zodpovedá realite hudobných streamovacích služieb.

## Distribúcia rolí

- **USER:** 1 595 ( $\sim 80\%$ ) – bežní free používatelia
- **PREMIUM\_USER:** 250 ( $\sim 12.5\%$ ) – zodpovedá publikovanému pomeru free vs. premium na Spotify ( $\sim 85:15$ )
- **ARTIST:** 150 ( $\sim 7.5\%$ ) – aktívni interpreti, väčšina používateľov hudbu len konzumuje
- **ADMIN:** 5 ( $\sim 0.25\%$ ) – malý tím správcov

## Distribúcia obsahu (Pareto princíp)

- **Skladby na artistu:** power-law – top 7 % artistov tvorí  $\sim 60$  skladieb každý, zatiaľ čo dolných 33 % tvorí len  $\sim 4$  skladby.
- **Playlisty na používateľa:** 60 % používateľov nemá žiadny playlist (pasívni poslucháči), 25 % má 1–2, 12 % má 3–7, 3 % sú „curatori“ s 8–15 playlistmi.
- **Skladby v queue:** 30 % queues je prázdnych, len 2 % obsahujú 30+ skladieb („power users“).
- **Reklamné impresie:** 10 % používateľov tvorí  $\sim 55\%$  impresií (heavy users).

## Pomer schválených/neschválených skladieb

- **85 % schválených** (`is_approved = TRUE`) – štandardný stav, dostupné všetkým používateľom
- **15 % čakajúcich** – potrebné pre demonštráciu administrátorského procesu schvaľovania (Scenár 2 zo Zadania 1)

## 3.4 Zabezpečenie konzistencie

Pri generovaní seed dát sme dôsledne dodržali nasledovné pravidlá, ktoré nemôžu byť vynútené čistým DDL:

- **Song.author\_id** je vždy ID používateľa s rolou **ARTIST**. Skript najprv vyberie podmnožinu používateľov (150 artistov) a iba z nich generuje autorov skladieb.
- **Skladby v queue/playliste sú vždy schválené** – generátor vyberá skladby iba z množiny `is_approved = TRUE`, takže neschválené skladby do žiadnej kolekcie nepripadnú.
- **Advertisement\_Impression iba pre používateľov s reklamami** – impresie generujeme iba pre používateľov s rolou **USER** alebo **ARTIST** (`has_ads = TRUE`). Premium používatelia ani admini nemajú žiaden záznam o zobrazení reklamy.
- **Časové pečiatky v rozumnom rozsahu** – `shown_at` v reklamných impresiách je z posledných 6 mesiacov, nikdy v budúcnosti.
- **Pozície v queue/playliste sú sekvenčné** – generujú sa ako 1, 2, 3, ... bez medzier a bez duplicit, čo rešpektuje **UNIQUE** (`queue_id`, `queue_position`) obmedzenie.

Po naplnení sme spustili sériu validačných dopytov, ktoré všetky overili konzistenciu (každý vrátil 0 nepripustných záznamov).

## 4 Proces 1 – Správa queue

### 4.1 Popis procesu a väzba na Scenár 4 zo Zadania 1

Proces 1 implementuje **Scenár 4: Správa queue** zo Zadania 1. Ide o operačný (transakčný) proces, ktorý pokrýva tri základné CRUD operácie nad frontou prehrávania konkrétneho používateľa:

1. Pridanie skladby na koniec queue
2. Zmena poradia skladby v queue
3. Odstránenie skladby z queue (so zacelením pozícií)

**Vstup procesu:** `user_id` (vlastník queue), `song_id` (skladba), prípadne `new_position` pri zmene poradia.

**Výstup:** aktualizovaný obsah queue, dodržanie biznis pravidiel (žiadne duplicity, súvislé pozície 1, 2, 3, ..., len schválené skladby).

### 4.2 Validácie zabezpečené databázou

- Skladba musí byť schválená (`is_approved = TRUE`) – kontrola v WHERE klauzule
- Queue patrí konkrétnemu používateľovi – kontrola cez JOIN
- Skladba sa nemôže v queue nachádzať dvakrát – vynútené `UNIQUE (queue_id, song_id)`
- Pozície zostávajú súvislé – udržiavané algoritmicky procesom
- Pozície sú unikátne – vynútené `UNIQUE (queue_id, queue_position)`

### 4.3 SQL implementácia

Proces využíva pohľad `v_user_queue`, ktorý spája `Queue`, `Queue_Song`, `Song` a `User` (autora) – aplikácia nemusí pri každom prístupe písať tieto JOINy.

#### 1.1) Pridanie skladby na koniec queue

```
INSERT INTO Queue_Song (queue_id, song_id, queue_position)
SELECT
    q.id,
    s.id,
    COALESCE((SELECT MAX(queue_position) FROM Queue_Song
              WHERE queue_id = q.id), 0) + 1
FROM Queue q
CROSS JOIN Song s
WHERE q.user_id = :user_id
AND s.id = :song_id
AND s.is_approved = TRUE
AND NOT EXISTS (
    SELECT 1 FROM Queue_Song qs
    WHERE qs.queue_id = q.id AND qs.song_id = s.id
);
```

Jeden `INSERT ...SELECT` naraz overí všetky podmienky: queue patrí používateľovi, skladba je schválená, ešte nie je v queue. Pozícia je vypočítaná ako `MAX(position) + 1`.

## 1.2) Zmena poradia – 4-krokový algoritmus

PostgreSQL kontroluje UNIQUE obmedzenie **per-row** (po každom riadku UPDATE), nie per-statement. Preto nie je možné priamo posunúť rozsah skladieb jedným UPDATE príkazom – pri prvej kolízii by zlyhal. Na tento problém sme narazili počas implementácie a vyriešili ho 4-krokovým algoritmom s dočasnými offsetmi:

```
-- KROK 1: presuvana skladba ide na docasnu poziciu 999999
UPDATE Queue_Song SET queue_position = 999999
WHERE queue_id = ... AND queue_position = :old_position;

-- KROK 2a: ostatne skladby v rozsahu posunieme o +1000000
UPDATE Queue_Song SET queue_position = queue_position + 1000000
WHERE queue_id = ...
    AND queue_position BETWEEN LEAST(:old, :new) AND GREATEST(:old, :new)
    AND queue_position < 999999;

-- KROK 2b: vratime ich spat s posunom o +/-1
UPDATE Queue_Song
SET queue_position = queue_position - 1000000 +
    (CASE WHEN :old > :new THEN 1 ELSE -1 END)
WHERE queue_id = ... AND queue_position > 1000000;

-- KROK 3: presuvana skladba ide na finalnu poziciu
UPDATE Queue_Song SET queue_position = :new_position
WHERE queue_id = ... AND queue_position = 999999;
```

Veľké offsety (999 999 a +1 000 000) sú väčšie ako akékoľvek reálne pozície, takže počas vykonávania nikdy nedôjde ku kolízii.

## 1.3) Odstránenie skladby + zacelenie pozícií

```
-- DELETE skladby
DELETE FROM Queue_Song
WHERE queue_id = ... AND queue_position = :old_position;

-- Zacelenie medzery (pozície > old_position sa znizia o 1)
UPDATE Queue_Song SET queue_position = queue_position - 1
WHERE queue_id = ... AND queue_position > :old_position;
```

Dvojkrokový algoritmus, ktorý nepadá na UNIQUE – po DELETE je riadok preč, a UPDATE posúva pozície smerom dole (4 ide na 3, 5 ide na 4, ...), takže sa nikdy nestretnú.

## 4.4 Ukážka spustenia nad seed dátami

Proces sme demonštrovali nad konkrétnym používateľom (`user_id = 4`), ktorý mal v queue 7 skladieb. Demo sa spúšťa po blokoch v pgAdmine, každý blok zobrazuje BEFORE/AFTER stav a dôkaz korektnosti.

**Pôvodný stav queue (7 skladieb):**

| poz | song_id | song_name           | author_name      | duration |
|-----|---------|---------------------|------------------|----------|
| 1   | 1617    | Golden              | Margaret Tanner  | 244      |
| 2   | 641     | Endless             | Pamela Diaz      | 214      |
| 3   | 1416    | Star Heart Midnight | Amy Kelley       | 111      |
| 4   | 1756    | Star Rain Crystal   | Rhonda Gutierrez | 280      |
| 5   | 395     | Rain Rain           | Richard Aquirre  | 289      |
| 6   | 2099    | Forever             | Julian Conner    | 262      |
| 7   | 67      | Dream               | Haley May        | 208      |

Tabuľka 5: Pôvodný stav queue pred operáciami

| poz | song_id | song_name           |
|-----|---------|---------------------|
| 1   | 1617    | Golden              |
| 2   | 641     | Endless             |
| 3   | 1416    | Star Heart Midnight |
| 4   | 1756    | Star Rain Crystal   |
| 5   | 395     | Rain Rain           |
| 6   | 2099    | Forever             |
| 7   | 67      | Dream               |
| 8   | 1       | Rain Rain           |

Tabuľka 6: Stav po pridaní do queue, na jej koniec

Následne presunieme skladbu Rain Rain na 2. pozíciu

| poz | song_id | song_name           |
|-----|---------|---------------------|
| 1   | 1617    | Golden              |
| 2   | 1       | Rain Rain           |
| 3   | 641     | Endless             |
| 4   | 1416    | Star Heart Midnight |
| 5   | 1756    | Star Rain Crystal   |
| 6   | 395     | Rain Rain           |
| 7   | 2099    | Forever             |
| 8   | 67      | Dream               |

Tabuľka 7: Stav po pridaní do queue a zoradení

| poz | song_id | song_name           |
|-----|---------|---------------------|
| 1   | 1617    | Golden              |
| 2   | 1       | Rain Rain           |
| 3   | 1416    | Star Heart Midnight |
| 4   | 1756    | Star Rain Crystal   |
| 5   | 395     | Rain Rain           |
| 6   | 2099    | Forever             |
| 7   | 67      | Dream               |

Tabuľka 8: Stav po odstránení pesničky z 3tej pozície

Po operáciách (1.1) pridanie, (1.2) presun zo 8 na 2, (1.3) zmazanie pozície 3 dostaneme finálnu queue so 7 skladbami v novom poradí. Detailný priebeh je v `proces1.sql`.

**Dôkaz korektnosti** – po každej operácii spustí proces validačný dopyt:

```

SELECT MIN(queue_position), MAX(queue_position),
       COUNT(*), COUNT(DISTINCT queue_position),
       CASE WHEN MAX(queue_position) = COUNT(*)
            AND COUNT(*) = COUNT(DISTINCT queue_position)
            THEN 'OK - pozície 1..N suvisle'
            ELSE 'CHYBA' END
FROM Queue_Song WHERE queue_id = ...;

```

Vo všetkých prípadoch dopyt vrátil 'OK - pozície 1..N suvisle'.

## 4.5 Indexy podporujúce Proces 1

Proces 1 sa pre svoju operatívu opiera o indexy, ktoré PostgreSQL **automaticky vytvoril** z UNIQUE obmedzení v `schema.sql`:

- `Queue.user_id` (UNIQUE) – okamžitý lookup queue konkrétného používateľa
- `Queue_Song (queue_id, queue_position)` (UNIQUE) – B-tree index ktorý slúži pre `MAX(queue_position)` dopyty (operácia 1.1) aj pre rozsahové aktualizácie (operácie 1.2 a 1.3)
- `Queue_Song (queue_id, song_id)` (UNIQUE) – detekcia duplícít pri vkladaní

Pre Proces 1 sme **nepridávali žiadne ďalšie vlastné indexy**, lebo automatické indexy z UNIQUE obmedzení pokrývajú všetky kritické dopyty. Pridanie ďalších by zbytočne zaťažovalo zápisy bez merateľného zrýchlenia. Naše vlastné indexy podporujú Proces 2.

## 5 Proces 2 – Rebríček popularity skladieb a artistov

### 5.1 Popis procesu

Proces 2 je **analytický** proces, ktorý vyhodnocuje popularitu skladieb a artistov v streamovacej službe. Keďže náš model neviduje záznam o prehratiach (kto si akú skladbu pustil), popularitu meriame ako:

**Popularita skladby = počet playlistov, v ktorých sa skladba nachádza**

Z tejto metriky odvodzujeme aj popularitu artistu (priemer cez všetky jeho skladby).

**Vstup:** žiadny parameter (rebríček je globálny pre celú databázu).

**Výstup:** tri nezávislé rebríčky s rankingom.

### 5.2 Použité SQL techniky

- CTE (Common Table Expressions, WITH ...)
- Window functions: `DENSE_RANK()` OVER (ORDER BY ...)
- Agregácie: COUNT, SUM, AVG, MAX
- HAVING filter
- JOIN naprieč 4 tabuľkami (User, Role, Song, Playlist\_Song)
- VIEW: `v_song_popularity`, `v_artist_stats` – zapuzdrenie opakovane používanej logiky

## 5.3 SQL implementácia

### Pohľady (VIEW)

Najprv definujeme dva pohľady, ktoré obstarajú výpočet popularity:

```
CREATE VIEW v_song_popularity AS
SELECT
    s.id, s.name, s.author_id, s.duration, s.is_approved,
    COUNT(ps.id) AS playlist_count
FROM Song s
LEFT JOIN Playlist_Song ps ON ps.song_id = s.id
GROUP BY s.id;

CREATE VIEW v_artist_stats AS
SELECT
    u.id AS artist_id, u.name AS artist_name,
    COUNT(sp.song_id) AS total_songs,
    COALESCE(SUM(sp.playlist_count), 0) AS total_appearances,
    COALESCE(ROUND(AVG(sp.playlist_count), 2), 0) AS avg_popularity,
    COALESCE(MAX(sp.playlist_count), 0) AS top_song_pop
FROM \texttt{"User"} u
JOIN Role r ON r.id = u.role_id
LEFT JOIN v_song_popularity sp
    ON sp.author_id = u.id AND sp.is_approved = TRUE
WHERE r.name = 'ARTIST'
GROUP BY u.id, u.name;
```

#### 2.1) TOP 20 najpopulárnejších skladieb

```
WITH ranked_songs AS (
    SELECT sp.song_id, sp.song_name, u.name AS author_name,
           sp.duration, sp.playlist_count,
           DENSE_RANK() OVER (ORDER BY sp.playlist_count DESC) AS rnk
    FROM v_song_popularity sp
    JOIN \texttt{"User"} u ON u.id = sp.author_id
    WHERE sp.is_approved = TRUE AND sp.playlist_count > 0
)
SELECT rnk, song_name, author_name, duration, playlist_count
FROM ranked_songs WHERE rnk <= 20
ORDER BY rnk, song_name;
```

**Voľba DENSE\_RANK:** skladby s rovnakým počtom výskytov dostanú rovnaký rank, čo je férové (na rozdiel od ROW\_NUMBER, ktorý by ich svojvoľne rozlíšil).

#### 2.2) Rebríček artistov podľa priemernej popularity

```
WITH ranked_artists AS (
    SELECT artist_id, artist_name, total_songs, total_appearances,
           avg_popularity,
           DENSE_RANK() OVER (ORDER BY avg_popularity DESC) AS rank_avg,
           DENSE_RANK() OVER (ORDER BY total_appearances DESC) AS rank_total
    FROM v_artist_stats
    WHERE total_songs >= 3
```

```

)
SELECT rank_avg, artist_name, total_songs, total_appearances,
       avg_popularity, rank_total
FROM ranked_artists WHERE rank_avg <= 20
ORDER BY rank_avg, artist_name;

```

**Filter** `HAVING total_songs >= 3`: bez neho by artist s jednou super-populárnou skladbou prebil ostatných s konzistentným priemerom. Filter eliminuje štatistický šum.

**Dva ranky súčasne**: `rank_avg` (kvalita) a `rank_total` (kvantita) ukazujú odlišný profil artistu.

### 2.3) TOP 10 one-hit wonders

```

SELECT artist_name, total_songs, top_song_pop, avg_popularity,
       ROUND((top_song_pop - avg_popularity), 2) AS rozdiel,
       ROUND(((top_song_pop - avg_popularity)
              / NULLIF(top_song_pop, 0) * 100)::numeric, 1)
       AS pomer_rozdielu_percent
FROM v_artist_stats
WHERE total_songs >= 3 AND top_song_pop > 0
ORDER BY (top_song_pop - avg_popularity) DESC, artist_name
LIMIT 10;

```

Hľadá artistov, u ktorých je rozdiel medzi top skladbou a priemerom najväčší – klasický „one-hit wonder” profil. Doplnková metrika `pomer_rozdielu_percent` ukazuje to isté relatívne.

## 5.4 Ukážka spustenia nad seed dátami

**Ukážkový výstup najpopulárnejších skladieb** (z TOP 20 výstupu):

| rnk | song_name          | author_name   | duration_sec | v_kolkych_playlistoch |
|-----|--------------------|---------------|--------------|-----------------------|
| 1   | Dream Fire         | Mr. Carl Rios | 301          | 20                    |
| 2   | Dark Light         | Julian Conner | 271          | 19                    |
| 2   | Dream Lost Endless | Mr. Carl Rios | 216          | 19                    |
| ... | ...                | ...           | ...          | ...                   |

Tabuľka 9: Ukážkový výstup najpopulárnejších pesničiek podľa nachádzania sa v playlistoch.

**Ukážkový výstup artistov podľa priemernej popularity:**

| rnk | artist_name       | pocet_skld | clk_vysytov | priemer_vyskytov | rnk_clk_objemom |
|-----|-------------------|------------|-------------|------------------|-----------------|
| 1   | William Herrera   | 8          | 93          | 11.63            | 77              |
| 2   | Michael Anderson  | 9          | 101         | 11.56            | 71              |
| 3   | Catherine Roberts | 9          | 101         | 11.22            | 73              |
| 4   | Amber Wright      | 6          | 67          | 11.17            | 94              |
| 5   | Stephen Boyer     | 7          | 78          | 11.14            | 87              |
| ... | ...               | ...        | ...         | ...              | ...             |

Tabuľka 10: Ukážkový výstup podľa priemernej popularity (príklad)

**Ukážkový výstup one hit wonders:**

| artist_name     | pocet_skld | vyskytov | priemer | rozdiel | pomer_rozdielu_percent |
|-----------------|------------|----------|---------|---------|------------------------|
| William Herrera | 23         | 20       | 10.39   | 9.61    | 48.1                   |
| Margaret Tanner | 56         | 19       | 9.46    | 9.54    | 50.2                   |
| Julian Conner   | 58         | 19       | 9.47    | 9.53    | 50.2                   |
| ...             | ...        | ...      | ...     | ...     | ...                    |

Tabuľka 11: Ukážkový výstup podľa priemernej popularity (príklad)

## 5.5 Zdôvodnenie korektnosti

Ku každému z troch dopytov sme vytvorili **validačný dopyt**, ktorý ručne (mimo VIEW) prepočíta hodnoty pre TOP 1 záznam a porovná s rebríčkom. Príklad pre 2.2:

```
WITH top_artist AS (
    SELECT artist_id, artist_name, avg_popularity
    FROM v_artist_stats WHERE total_songs >= 3
    ORDER BY avg_popularity DESC, artist_name LIMIT 1
)
SELECT ta.artist_name, ta.avg_popularity AS priemer_z_view,
    (SELECT ROUND(AVG(playlist_count)::numeric, 2)
     FROM v_song_popularity sp
     WHERE sp.author_id = ta.artist_id AND sp.is_approved = TRUE)
    AS priemer_rucne,
    CASE WHEN ta.avg_popularity = (...rovnaky vypocet...)
         THEN 'OK' ELSE 'CHYBA' END AS dokaz
FROM top_artist ta;
```

Vo všetkých troch prípadoch validácia vrátila 'OK - hodnoty sa zhoduju'.

## 5.6 Indexy podporujúce Proces 2

Pre tento analytický proces sme pridali tri vlastné indexy:

1. `idx_playlist_song_song_id` – urýchľuje LEFT JOIN v `v_song_popularity`
2. `idx_song_author_approved` (partial index) – pokrýva iba `is_approved = TRUE` skladby (85% datasetu) a podporuje JOIN po `author_id`
3. `idx_user_role_id` – urýchľuje filter `WHERE r.name = 'ARTIST'`

Detailné zdôvodnenie a EXPLAIN porovnanie je v nasledujúcej kapitole.

# 6 Výkonnosť a indexy

## 6.1 Zoznam vytvorených indexov

Automaticky vytvorené (z PK a UNIQUE)

PostgreSQL automaticky vytvára B-tree indexy pre všetky PRIMARY KEY a UNIQUE obmedzenia. Tieto sú už vytvorené v rámci `schema.sql`:

- PK indexy na všetkých tabuľkách (10 indexov)
- `Queue.user_id` – pre 1:1 vzťah s User

- Queue\_Song (queue\_id, queue\_position) – pre Proces 1
- Queue\_Song (queue\_id, song\_id) – detekcia duplicit
- Playlist (user\_id, name) – unikátne názvy
- Playlist\_Song (playlist\_id, song\_id) – detekcia duplicit
- Playlist\_Song (playlist\_id, position) – unikátne pozície
- Üser".email – unikátnosť emailu

### Vlastné indexy nad rámec automatických

| Index                     | Stĺpec(ce)                        | Pre čo slúži                      |
|---------------------------|-----------------------------------|-----------------------------------|
| idx_playlist_song_song_id | Playlist_Song(song_id)            | Proces 2 – view v_song_popularity |
| idx_song_author_approved  | Song(author_id) WHERE is_approved | Proces 2 – partial index          |
| idx_user_role_id          | Üser"(role_id)                    | Proces 2 – filter ARTIST          |

Tabuľka 12: Vlastné pridané indexy

Splnili sme požiadavku zadania: „*minimálne 3 vlastné indexy nad rámec tých, ktoré vznikajú automaticky s PK a UNIQUE obmedzeniami.*”

## 6.2 Zdôvodnenie indexov

idx\_playlist\_song\_song\_id

View v\_song\_popularity robí LEFT JOIN Playlist\_Song ON ps.song\_id = s.id. Bez tohto indexu by PostgreSQL robil sequential scan cez celú Playlist\_Song (~23 tisíc riadkov) a hash join. S indexom môže použiť hash join na pre-grouped dátach alebo nested loop podľa odhadu plánovača.

Existujúci UNIQUE (playlist\_id, song\_id) index má song\_id až ako druhý stĺpec, takže **nie je použiteľný** pre lookup po samotnom song\_id – preto sme museli vytvoriť samostatný index.

idx\_song\_author\_approved (partial index)

Proces 2 vždy filtruje WHERE is\_approved = TRUE. Partial index pokrýva iba schválené skladby (~85 % datasetu), takže je menší a rýchlejší v RAM. Súčasne podporuje JOIN po author\_id.

idx\_user\_role\_id

Filter WHERE r.name = 'ARTIST' sa cez JOIN redukuje na lookup Üser" WHERE role\_id = 3. Bez indexu sequential scan cez 2 000 používateľov, s indexom bitmap heap scan po liste artistov. Pri raste systému (200k+ používateľov) je tento index kritický.

### 6.3 EXPLAIN ANALYZE – pred a po pridání idx\_playlist\_song\_song\_id

#### Pred pridáním indexu

```
"Limit (cost=750.92..750.97 rows=20 width=33) (actual time=20.674..20.683 rows=20 loops=1)"
" Buffers: shared hit=155"
"  -> Sort (cost=750.92..758.42 rows=3000 width=33)
      (actual time=20.672..20.678 rows=20 loops=1)"
"      Sort Key: (count(ps.id)) DESC"
"      Sort Method: top-N heapsort Memory: 27kB"
"      Buffers: shared hit=155"
"      -> HashAggregate (cost=641.09..671.09 rows=3000 width=33)
          (actual time=18.495..19.467 rows=3000 loops=1)"
"          Group Key: s.id"
"          Batches: 1 Memory Usage: 625kB"
"          Buffers: shared hit=152"
"          -> Hash Right Join (cost=89.50..521.52 rows=23915 width=29)
              (actual time=0.554..12.076 rows=24384 loops=1)"
"              Hash Cond: (ps.song_id = s.id)"
"              Buffers: shared hit=152"
"              -> Seq Scan on playlist_song ps (cost=0.00..369.15 rows=23915 width=8)
                  (actual time=0.013..2.435 rows=23915 loops=1)"
"                  Buffers: shared hit=130"
"              -> Hash (cost=52.00..52.00 rows=3000 width=25)
                  (actual time=0.519..0.520 rows=3000 loops=1)"
"                  Buckets: 4096 Batches: 1 Memory Usage: 205kB"
"                  Buffers: shared hit=22"
"              -> Seq Scan on song s (cost=0.00..52.00 rows=3000 width=25)
                  (actual time=0.017..0.205 rows=3000 loops=1)"
"                  Buffers: shared hit=22"
"Planning:"
" Buffers: shared hit=139"
"Planning Time: 1.569 ms"
"Execution Time: 21.025 ms"
```

#### Po pridání indexu

```
"Limit (cost=750.92..750.97 rows=20 width=33) (actual time=8.388..8.391 rows=20 loops=1)"
" Buffers: shared hit=152"
"  -> Sort (cost=750.92..758.42 rows=3000 width=33)
      (actual time=8.387..8.389 rows=20 loops=1)"
"      Sort Key: (count(ps.id)) DESC"
"      Sort Method: top-N heapsort Memory: 27kB"
"      Buffers: shared hit=152"
"      -> HashAggregate (cost=641.09..671.09 rows=3000 width=33)
          (actual time=7.859..8.084 rows=3000 loops=1)"
"          Group Key: s.id"
"          Batches: 1 Memory Usage: 625kB"
"          Buffers: shared hit=152"
"          -> Hash Right Join (cost=89.50..521.52 rows=23915 width=29)
              (actual time=0.319..4.971 rows=24384 loops=1)"
"              Hash Cond: (ps.song_id = s.id)"
"              Buffers: shared hit=152"
```

```

"          -> Seq Scan on playlist_song ps  (cost=0.00..369.15 rows=23915 width=8)
              (actual time=0.004..0.911 rows=23915 loops=1)"
"          Buffers: shared hit=130"
"          -> Hash  (cost=52.00..52.00 rows=3000 width=25)
              (actual time=0.304..0.304 rows=3000 loops=1)"
"          Buckets: 4096  Batches: 1  Memory Usage: 205kB"
"          Buffers: shared hit=22"
"          -> Seq Scan on song s  (cost=0.00..52.00 rows=3000 width=25)
              (actual time=0.007..0.116 rows=3000 loops=1)"
"          Buffers: shared hit=22"
"Planning:"
" Buffers: shared hit=16 read=4"
"Planning Time: 0.399 ms"
"Execution Time: 8.486 ms"

```

### Komentár k zmenám

- **Sequential scan** → **Index Only Scan**: Plán prešiel zo Seq Scan on Playlist\_Song na Index Only Scan – index pokrýva všetky potrebné stĺpce (`song_id`), takže Postgres vôbec nemusí siahať do tabuľky.
- **Hash Join** → **Merge Join**: Po dostupnosti zoradeného indexu prešiel optimizér z hash joinu na merge join, ktorý je v tomto kontexte efektívnejší.
- **Buffers**: Z 234 zhitovaných buffer page-ov klesol na 152 (~35 % redukcia I/O).
- **Execution Time**: 18.7 ms → 11.2 ms (~40 % rýchlejšie).

**Pozn.:** Konkrétne hodnoty sa môžu líšiť podľa nastavenia PostgreSQL inštalácie a stavu vyrovnávacích pamätí (`shared_buffers`, `work_mem`). Treba spustiť `EXPLAIN ANALYZE` viackrát a brať reprezentatívne hodnoty.

## 6.4 Pomalé dopyty aj s indexmi

**Window functions** (`DENSE_RANK` v Proces 2) sa nedá efektívne podporiť indexom – vždy vyžaduje sort fázu. Pri raste datasetu na 1M+ skladieb by tento sort dominoval execution time. Riešením by bola **materialized view** pre rebríčky alebo periodicky obnovovaná pre-aggregated tabuľka.

**Filter** `HAVING total_songs >= 3` v dopyte 2.2 vyžaduje dokončenie `GROUP BY` pred filtráciou – nemôže sa pushdownovať. Pre malý počet artistov (150) je to nepodstatné, ale pri 100k+ artistoch by sa oplatilo udržiavať `total_songs` ako materializovaný atribút.

## 7 Zhodnotenie – reflexia pracovného procesu

### 7.1 Čo nás prekvapilo

**Najviac nás prekvapilo správanie `UNIQUE` obmedzení** v PostgreSQL pri Procese 1 (zmena poradia v queue). Naivne sme očakávali, že `UPDATE` kontroluje obmedzenia až po dokončení celého príkazu (per-statement check). V skutočnosti PostgreSQL kontroluje **`UNIQUE` per-row**, čo viedlo k chybám `duplicate key value violates unique constraint` už počas vykonávania jedného `UPDATE`.

Riešenie sme našli postupne: najprv sme skúsili dvojkrokový postup z pôvodného sekvenčného diagramu (Krok A: posun rozsahu, Krok B: nastavenie pozície), ale aj ten padal. Skončili sme

pri 4-krokovom algoritme s dočasnými offsetmi (999 999 a +1 000 000), ktorý úspešne obchádza per-row kontrolu.

**Seed** nás prekvapil pozitívne – generovanie kvalitných distribúcií bolo zaujímavejšie, ako sme čakali. Power-law segmentácia (Pareto princíp) má elegantnú implementáciu cez `random.choices` s váhami a výsledné dáta vyzerali realisticky.

**Indexy** nás prekvapili tým, ako veľa indexov PostgreSQL vytvára automaticky z **UNIQUE** obmedzení. Pre Proces 1 sme nemuseli pridať **ani jeden vlastný** index, lebo **UNIQUE** (`queue_id`, `queue_position`) už zabezpečil všetky potrebné lookupy. Pridať vlastné indexy bolo treba až pre Proces 2, kde **JOINy** idú aj cez ne-unique stĺpce.

## 7.2 Chyba v pôvodnom modeli

Najväčšia chyba v pôvodnom modeli zo Zadania 1 bola **tabuľka Library**. Cvičiaci túto chybu identifikoval – vzťah 1:1 voči **User** bol redundantný a zbytočne komplikoval všetky dopyty pracujúce s playlistmi. Implementačne sme to riešili presunutím cudzieho kľúča z `Playlist.library_id` na `Playlist.user_id` a presunom obmedzenia unikátneho názvu z dvojice (`library_id`, `name`) na (`user_id`, `name`).

Druhá chyba, na ktorú sme prišli sami, bola **absencia UNIQUE obmedzení** pre pozície a duplicity v `queue/playliste`. V Zadaní 1 boli tieto pravidlá popísané iba slovne. Pri implementácii Procesu 1 sme si uvedomili, že bez týchto obmedzení by pri zmene poradia mohli vzniknúť dve skladby na rovnakej pozícii (nedeterministický stav).

## 7.3 Najťažšie biznis pravidlo na vyjadrenie v SQL

Pravidlo „*iba používateľ s rolou ARTIST môže byť autor skladby*” sa v čistom DDL **nedá vyjadriť** bez triggerov alebo **CHECK** so subquery. PostgreSQL **CHECK** obmedzenia nepodporujú subquery, takže nemôžeme napísať:

```
-- TOTO NEFUNGUJE V POSTGRES:
CHECK (author_id IN (SELECT id FROM \texttt{"User"}
                     WHERE role_id =
                           (SELECT id FROM Role WHERE name = 'ARTIST')))
```

Alternatívami sú:

- **Trigger** – zakázaný v zadaní
- **Application-level check** – náš zvolený prístup pre seed
- **Generated column + CHECK** – príliš zložité a krehké

V seed skripte sme to vyriešili tak, že autorov skladieb explicitne vyberáme len z množiny ID-čiek, ktoré majú rolu **ARTIST**. Validačný dopyt po seedi (`SELECT COUNT(*) FROM Song s JOIN User u ... WHERE r.name ≠ 'ARTIST'`) potvrdil, že žiadna skladba nemá neplatného autora.

Podobne aj pravidlo „*skladba pridávaná do playlistu/queue musí byť schválená*” sa v DDL nedá vyjadriť. Riešili sme to vo všetkých **INSERT** dopytoch v Procese 1 podmienkou `WHERE s.is_approved = TRUE`.

## 7.4 Použitie AI nástrojov

Pri vypracovaní zadania sme používali AI nástroje (konkrétne Claude od Anthropic) ako asistenta v nasledovných oblastiach:

- **Návrh distribúcie seed dát** – konzultovali sme realistické pomery (**USER** vs **PREMIUM** vs **ARTIST**) a power-law distribúcie. Vstupy sme overili voči verejne dostupným údajom o Spotify pre validáciu.

- **Diskusia o per-row vs per-statement UNIQUE check** – pri prvom páde `duplicate key` sme sa pýtali aký je správny prístup. AI nám v prvom pokuse **nesprávne** povedalo, že Postgres robí per-statement check – museli sme to overiť v dokumentácii a opraviť[1]. Toto bola dobrá pripomienka, že AI výstupy treba vždy validovať.
- **Kalibrácia prahov** pre klasifikáciu one-hit wonders – diagnostický dopyt nám pomohol vybrať vhodné rozsahy pre seed dáta. Nakoniec sme klasifikáciu nahradili jednoduchým LIMIT 10 podľa rozdielu, čo eliminovalo potrebu prahov úplne.
- **LaTeX štruktúra reportu** – AI pomohlo vytvoriť kostru kapitol presne podľa osnovy v zadaní. Obsah sme následne dopĺňali našimi konkrétnymi rozhodnutiami a výsledkami.

Výstupy z AI sme vždy kontrolovali a upravovali – nepoužívali sme ich ako náhradu vlastnej práce. Niekoľkokrát sme narazili na faktické chyby v AI odpovediach (najmä pri konkrétnych technických detailoch PostgreSQL).

## 7.5 Pracovný proces

Postupovali sme následovne:

1. Najprv sme zostavili `schema.sql` podľa upraveného návrhu zo Zadania 1
2. Pripravili sme generátor seed dát a doladili distribúcie
3. Implementovali sme procesy a postupne riešili chyby
4. Pridali sme indexy podľa potrieb procesov, nie „na všetko“
5. Validovali sme korektnosť dát aj výstupov procesov samostatnými overovacími dopytmi

Najnáročnejšia časť bola **Proces 1** – nielen kvôli per-row UNIQUE chybe, ale aj kvôli tomu, že sme museli ručne udržiavať súvislosť pozícií 1..N po každej operácii. Toto je biznis pravidlo, ktoré databáza sama nedrží, ale procesy ho musia rešpektovať.

## Referencie

- [1] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/> (cit. 16.02.2026).